

Model Training with Ultralytics YOLO



Introduction

Training a [deep learning](#) model involves feeding it data and adjusting its parameters so that it can make accurate predictions. Train mode in Ultralytics YOLO26 is engineered for effective and efficient training of object detection models, fully utilizing modern hardware capabilities. This guide aims to cover all the details you need to get started with training your own models using YOLO26's robust set of features.

Ask AI 



How to Train Ultralytics YOLOv8 models on Your Custc

Ultralytics



Watch on

Watch: How to Train a YOLO model on Your Custom Dataset in Google Colab.

YouTube video player

Why Choose Ultralytics YOLO for Training?

Here are some compelling reasons to opt for YOLO26's Train mode:

- **Efficiency:** Make the most out of your hardware, whether you're on a single-GPU setup or scaling across multiple GPUs.
- **Versatility:** Train on custom datasets in addition to readily available ones like COCO, VOC, and ImageNet.
- **User-Friendly:** Simple yet powerful CLI and Python interfaces for a straightforward training experience.
- **Hyperparameter Flexibility:** A broad range of customizable hyperparameters to fine-tune model performance. For deeper control, you can [customize the trainer](#) itself.
- **Cloud Training:** Train on cloud GPUs through [Ultralytics Platform](#) with real-time metrics and automatic checkpointing.

Key Features of Train Mode

The following are some notable features of YOLO26's Train mode:

- **Automatic Dataset Download:** Standard datasets like COCO, VOC, and ImageNet are downloaded automatically on first use.
- **Multi-GPU Support:** Scale your training efforts seamlessly across multiple GPUs to expedite the process.
- **Hyperparameter Configuration:** The option to modify hyperparameters through YAML configuration files or CLI arguments.
- **Visualization and Monitoring:** Real-time tracking of training metrics and visualization of the learning process for better insights.



Tip

- YOLO26 datasets like COCO, VOC, ImageNet, and many others automatically download on first use, i.e., `yolo train data=coco.yaml`

Usage Examples

Train YOLO26n on the COCO8 dataset for 100 `epochs` at image size 640. The training device can be specified using the `device` argument. If no argument is passed, GPU `device=0` will be used when available; otherwise `device='cpu'` will be used. See the Arguments section below for a full list of training arguments.



Windows Multi-Processing Error

On Windows, you may receive a `RuntimeError` when launching the training as a script. Add an `if __name__ == "__main__":` block before your training code to resolve it.



Single-GPU and CPU Training Example

Device is determined automatically. If a GPU is available, it will be used (default CUDA device 0); otherwise training will start on CPU.

Python

```
from ultralytics import YOLO

# Load a model
model = YOLO("yolo26n.yaml") # build a new model from YAML
model = YOLO("yolo26n.pt") # load a pretrained model (recommended for training)
model = YOLO("yolo26n.yaml").load("yolo26n.pt") # build from YAML and transfer weights

# Train the model
results = model.train(data="coco8.yaml", epochs=100, imgsz=640)
```

CLI

```
# Build a new model from YAML and start training from scratch
yolo detect train data=coco8.yaml model=yolo26n.yaml epochs=100 imgsz=640

# Start training from a pretrained *.pt model
yolo detect train data=coco8.yaml model=yolo26n.pt epochs=100 imgsz=640

# Build a new model from YAML, transfer pretrained weights to it and start training
yolo detect train data=coco8.yaml model=yolo26n.yaml pretrained=yolo26n.pt
epochs=100 imgsz=640
```

Multi-GPU Training

Multi-GPU training allows for more efficient utilization of available hardware resources by distributing the training load across multiple GPUs. This feature is available through both the Python API and the command-line interface. To enable multi-GPU training, specify the GPU device IDs you wish to use.



Multi-GPU Training Example

To train with 2 GPUs, CUDA devices 0 and 1 use the following commands. Expand to additional GPUs as required.

Python

```

from ultralytics import YOLO

# Load a model
model = YOLO("yolo26n.pt") # load a pretrained model (recommended for training)

# Train the model with 2 GPUs
results = model.train(data="coco8.yaml", epochs=100, imgsz=640, device=[0, 1])

# Train the model with the two most idle GPUs
results = model.train(data="coco8.yaml", epochs=100, imgsz=640, device=[-1, -1])

```

CLI

```

# Start training from a pretrained *.pt model using GPUs 0 and 1
yolo detect train data=coco8.yaml model=yolo26n.pt epochs=100 imgsz=640 device=0,1

# Use the two most idle GPUs
yolo detect train data=coco8.yaml model=yolo26n.pt epochs=100 imgsz=640 device=-1,-1

```



Multi-GPU Training with Custom Code

When you specify multiple devices (e.g., `device=[0, 1]`), Ultralytics internally spawns a new trainer instance and executes `torch.distributed.run` under the hood. This works seamlessly for standard CLI usage and unmodified Python scripts.

However, if your script contains custom components—such as a custom trainer, validator, dataset, or augmentation pipeline—these objects cannot be automatically serialized and transferred to the DDP subprocesses. In this case, you must launch your script directly with `torch.distributed.run`:

```
python -m torch.distributed.run --nproc_per_node 2 your_training_script.py
```

Idle GPU Training

Idle GPU Training enables automatic selection of the least utilized GPUs in multi-GPU systems, optimizing resource usage without manual GPU selection. This feature identifies available GPUs based on utilization metrics and VRAM availability.



Idle GPU Training Example

To automatically select and use the most idle GPU(s) for training, use the `-1` device parameter. This is particularly useful in shared computing environments or servers with multiple users.

Python

```
from ultralytics import YOLO

# Load a model
model = YOLO("yolo26n.pt") # load a pretrained model (recommended for training)

# Train using the single most idle GPU
results = model.train(data="coco8.yaml", epochs=100, imgsz=640, device=-1)

# Train using the two most idle GPUs
results = model.train(data="coco8.yaml", epochs=100, imgsz=640, device=[-1, -1])
```

CLI

```
# Start training using the single most idle GPU
yolo detect train data=coco8.yaml model=yolo26n.pt epochs=100 imgsz=640 device=-1

# Start training using the two most idle GPUs
yolo detect train data=coco8.yaml model=yolo26n.pt epochs=100 imgsz=640 device=-1,-1
```

The auto-selection algorithm prioritizes GPUs with:

1. Lower current utilization percentages
2. Higher available memory (free VRAM)
3. Lower temperature and power consumption

This feature is especially valuable in shared computing environments or when running multiple training jobs across different models. It automatically adapts to changing system conditions, ensuring optimal resource allocation without manual intervention.

Apple Silicon MPS Training

With the support for Apple silicon chips integrated in the Ultralytics YOLO models, it's now possible to train your models on devices utilizing the powerful Metal Performance Shaders (MPS) framework. The MPS offers a high-performance way of executing computation and image processing tasks on Apple's custom silicon.

To enable training on Apple silicon chips, you should specify 'mps' as your device when initiating the training process. Below is an example of how you could do this in Python and via the command line:



MPS Training Example

Python

```
from ultralytics import YOLO

# Load a model
model = YOLO("yolo26n.pt") # load a pretrained model (recommended for training)

# Train the model with MPS
results = model.train(data="coco8.yaml", epochs=100, imgsz=640, device="mps")
```

CLI

```
# Start training from a pretrained *.pt model using MPS
yolo detect train data=coco8.yaml model=yolo26n.pt epochs=100 imgsz=640 device=mps
```

While leveraging the computational power of the Apple silicon chips, this enables more efficient processing of the training tasks. For more detailed guidance and advanced configuration options, please refer to the [PyTorch MPS documentation](#).

Resuming Interrupted Trainings

Resuming training from a previously saved state is a crucial feature when working with deep learning models. This can come in handy in various scenarios, like when the training process has been unexpectedly interrupted, or when you wish to continue training a model with new data or for more epochs.

When training is resumed, Ultralytics YOLO loads the weights from the last saved model and also restores the optimizer state, [learning rate](#) scheduler, and the epoch number. This allows you to continue the training process seamlessly from where it was left off.

You can easily resume training in Ultralytics YOLO by setting the `resume` argument to `True` when calling the `train` method, and specifying the path to the `.pt` file containing the partially trained model weights.

Below is an example of how to resume an interrupted training using Python and via the command line:



Resume Training Example

Python

```
from ultralytics import YOLO

# Load a model
model = YOLO("path/to/last.pt") # load a partially trained model

# Resume training
results = model.train(resume=True)
```

CLI

```
# Resume an interrupted training
yolo train resume model=path/to/last.pt
```

By setting `resume=True`, the `train` function will continue training from where it left off, using the state stored in the 'path/to/last.pt' file. If the `resume` argument is omitted or set to `False`, the `train` function will start a new training session.

Remember that checkpoints are saved at the end of every epoch by default, or at fixed intervals using the `save_period` argument, so you must complete at least 1 epoch to resume a training run.

Train Settings

The training settings for YOLO models encompass various hyperparameters and configurations used during the training process. These settings influence the model's performance, speed, and [accuracy](#). Key training settings include batch size, learning rate, momentum, and weight decay. Additionally, the choice of optimizer, [loss function](#), and training dataset composition can impact the training process. Careful tuning and experimentation with these settings are crucial for optimizing performance.

MuSGD Optimizer

In YOLO26, **MuSGD** is a hybrid optimizer that combines standard **SGD** updates with **Muon-style orthogonalized updates**.

It is **recommended for longer YOLO26 training runs and larger datasets**, where orthogonalized Muon updates can help stabilize optimization.

Only parameters with `param.ndim >= 2` (such as convolutional weights) receive the Muon style update together with SGD, while lower dimensional parameters like batch normalization layers and bias terms remain on standard SGD.

When `optimizer=auto` is used, Ultralytics automatically selects **MuSGD** for longer training runs (typically when iterations > 10000). For shorter runs, the trainer falls back to **AdamW**.

Example usage:

```

yolo train model=yolo26n.pt data=coco8.yaml optimizer=MusGD

```

See the implementation in `ultralytics/optim/muon.py` and the optimizer auto-selection logic in `BaseTrainer.build_optimizer`.

Argument	Type	Default	Description
<code>model</code>	<code>str</code>	<code>None</code>	Specifies the model file for training. Accepts a path to either a <code>.pt</code> pretrained model or a <code>.yaml</code> configuration file. Essential for defining the model structure or initializing weights.
<code>data</code>	<code>str</code>	<code>None</code>	Path to the dataset configuration file (e.g., <code>coco8.yaml</code>). This file contains

Argument	Type	Default	Description
			dataset-specific parameters, including paths to training and validation data , class names, and number of classes.
<code>epochs</code>	<code>int</code>	<code>100</code>	Total number of training epochs. Each epoch represents a full pass over the entire dataset. Adjusting this value can affect training duration and model performance.
<code>time</code>	<code>float</code>	<code>None</code>	Maximum training time in hours. If set, this overrides the <code>epochs</code> argument, allowing training to automatically stop after the specified duration. Useful for time-constrained training scenarios.
<code>patience</code>	<code>int</code>	<code>100</code>	Number of epochs to wait without improvement in validation metrics before early stopping the training. Helps prevent overfitting by stopping training when performance plateaus.
<code>batch</code>	<code>int or float</code>	<code>16</code>	Batch size , with three modes: set as an integer (e.g., <code>batch=16</code>), auto mode for 60% GPU memory utilization (<code>batch=-1</code>), or auto mode with specified utilization fraction (<code>batch=0.70</code>).
<code>imgsz</code>	<code>int</code>	<code>640</code>	Target image size for training. Images are resized to squares with sides equal to the specified value (if <code>rect=False</code>), preserving aspect ratio for YOLO models but not RT-DETR. Affects model accuracy and computational complexity.
<code>save</code>	<code>bool</code>	<code>True</code>	Enables saving of training checkpoints and final model weights. Useful for resuming training or model deployment .
<code>save_period</code>	<code>int</code>	<code>-1</code>	Frequency of saving model checkpoints, specified in epochs. A value of <code>-1</code> disables this feature. Useful for saving interim models during long training sessions.
<code>cache</code>	<code>bool</code>	<code>False</code>	Enables caching of dataset images in memory (<code>True/ram</code>), on disk (<code>disk</code>), or disables it (<code>False</code>). Improves training speed by reducing disk I/O at the cost of increased memory usage.

Argument	Type	Default	Description
<code>device</code>	<code>int</code> or <code>str</code> or <code>list</code>	<code>None</code>	Specifies the computational device(s) for training: a single GPU (<code>device=0</code>), multiple GPUs (<code>device=[0,1]</code>), CPU (<code>device=cpu</code>), MPS for Apple silicon (<code>device=mps</code>), Huawei Ascend NPU (<code>device=npu</code> or <code>device=npu:0</code>), or auto-selection of most idle GPU (<code>device=-1</code>) or multiple idle GPUs (<code>device=[-1,-1]</code>)
<code>workers</code>	<code>int</code>	<code>8</code>	Number of worker threads for data loading (per <code>RANK</code> if Multi-GPU training). Influences the speed of data preprocessing and feeding into the model, especially useful in multi-GPU setups.
<code>project</code>	<code>str</code>	<code>None</code>	Name of the project directory where training outputs are saved. Allows for organized storage of different experiments.
<code>name</code>	<code>str</code>	<code>None</code>	Name of the training run. Used for creating a subdirectory within the project folder, where training logs and outputs are stored.
<code>exist_ok</code>	<code>bool</code>	<code>False</code>	If <code>True</code> , allows overwriting of an existing project/name directory. Useful for iterative experimentation without needing to manually clear previous outputs.
<code>pretrained</code>	<code>bool</code> or <code>str</code>	<code>True</code>	Determines whether to start training from pretrained weights. Can be a boolean value or a string path to weights to load. <code>pretrained=False</code> trains from randomly initialized weights while keeping the model architecture.
<code>optimizer</code>	<code>str</code>	<code>'auto'</code>	Choice of optimizer for training. Options include <code>SGD</code> , <code>MuSGD</code> , <code>Adam</code> , <code>Adamax</code> , <code>AdamW</code> , <code>NAdam</code> , <code>RAdam</code> , <code>RMSProp</code> , or <code>auto</code> for automatic selection based on model configuration. Affects convergence speed and stability.
<code>seed</code>	<code>int</code>	<code>0</code>	Sets the random seed for training, ensuring reproducibility of results across runs with the same configurations.

Argument	Type	Default	Description
<code>deterministic</code>	<code>bool</code>	<code>True</code>	Forces deterministic algorithm use, ensuring reproducibility but may affect performance and speed due to the restriction on non-deterministic algorithms.
<code>verbose</code>	<code>bool</code>	<code>True</code>	Enables verbose output during training, displaying progress bars, per-epoch metrics, and additional training information in the console.
<code>single_cls</code>	<code>bool</code>	<code>False</code>	Treats all classes in multi-class datasets as a single class during training. Useful for binary classification tasks or when focusing on object presence rather than classification.
<code>classes</code>	<code>list[int]</code>	<code>None</code>	Specifies a list of class IDs to train on. Useful for filtering out and focusing only on certain classes during training.
<code>rect</code>	<code>bool</code>	<code>False</code>	Enables minimum padding strategy—images in a batch are minimally padded to reach a common size, with the longest side equal to <code>imgsz</code> . Can improve efficiency and speed but may affect model accuracy.
<code>multi_scale</code>	<code>float</code>	<code>0.0</code>	Randomly vary <code>imgsz</code> each batch by $\pm \text{multi_scale}$ (e.g. <code>0.25</code> -> <code>0.75x</code> to <code>1.25x</code>), rounding to model stride multiples; <code>0.0</code> disables multi-scale training.
<code>cos_lr</code>	<code>bool</code>	<code>False</code>	Utilizes a cosine learning rate scheduler, adjusting the learning rate following a cosine curve over epochs. Helps in managing learning rate for better convergence.
<code>close_mosaic</code>	<code>int</code>	<code>10</code>	Disables mosaic data augmentation in the last N epochs to stabilize training before completion. Setting to 0 disables this feature.
<code>resume</code>	<code>bool</code>	<code>False</code>	Resumes training from the last saved checkpoint. Automatically loads model weights, optimizer state, and epoch count, continuing training seamlessly.
<code>amp</code>	<code>bool</code>	<code>True</code>	Enables Automatic Mixed Precision (AMP) training, reducing memory

Argument	Type	Default	Description
			usage and possibly speeding up training with minimal impact on accuracy.
<code>fraction</code>	<code>float</code>	<code>1.0</code>	Specifies the fraction of the dataset to use for training. Allows for training on a subset of the full dataset, useful for experiments or when resources are limited.
<code>profile</code>	<code>bool</code>	<code>False</code>	Enables profiling of ONNX and TensorRT speeds during training, useful for optimizing model deployment.
<code>freeze</code>	<code>int or list</code>	<code>None</code>	Freezes the first N layers of the model or specified layers by index, reducing the number of trainable parameters. Useful for fine-tuning or transfer learning .
<code>lr0</code>	<code>float</code>	<code>0.01</code>	Initial learning rate (i.e. SGD=1E-2, Adam=1E-3). Adjusting this value is crucial for the optimization process, influencing how rapidly model weights are updated.
<code>lrf</code>	<code>float</code>	<code>0.01</code>	Final learning rate as a fraction of the initial rate = ($lr0 * lrf$), used in conjunction with schedulers to adjust the learning rate over time.
<code>momentum</code>	<code>float</code>	<code>0.937</code>	Momentum factor for SGD or beta1 for Adam optimizers , influencing the incorporation of past gradients in the current update.
<code>weight_decay</code>	<code>float</code>	<code>0.0005</code>	L2 regularization term, penalizing large weights to prevent overfitting.
<code>warmup_epochs</code>	<code>float</code>	<code>3.0</code>	Number of epochs for learning rate warmup, gradually increasing the learning rate from a low value to the initial learning rate to stabilize training early on.
<code>warmup_momentum</code>	<code>float</code>	<code>0.8</code>	Initial momentum for warmup phase, gradually adjusting to the set momentum over the warmup period.

Argument	Type	Default	Description
warmup_bias_lr	float	0.1	Learning rate for bias parameters during the warmup phase, helping stabilize model training in the initial epochs.
box	float	7.5	Weight of the box loss component in the loss function , influencing how much emphasis is placed on accurately predicting bounding box coordinates.
cls	float	0.5	Weight of the classification loss in the total loss function, affecting the importance of correct class prediction relative to other components.
cls_pw	float	0.0	Power for class weighting to handle class imbalance using inverse class frequency. 0.0 disables class weighting, 1.0 applies full inverse frequency weighting. Values between 0 and 1 provide partial weighting.
dfl	float	1.5	Weight of the distribution focal loss, used in certain YOLO versions for fine-grained classification.
pose	float	12.0	Weight of the pose loss in models trained for pose estimation, influencing the emphasis on accurately predicting pose keypoints.
kobj	float	1.0	Weight of the keypoint objectness loss in pose estimation models, balancing detection confidence with pose accuracy.
rle	float	1.0	Weight of the residual log-likelihood estimation loss in pose estimation models, affecting the precision of keypoint localization.
angle	float	1.0	Weight of the angle loss in obb models, affecting the precision of oriented bounding box angle predictions.
nbs	int	64	Nominal batch size for normalization of loss.
overlap_mask	bool	True	Determines whether object masks should be merged into a single mask

Argument	Type	Default	Description
			for training, or kept separate for each object. In case of overlap, the smaller mask is overlaid on top of the larger mask during merge.
mask_ratio	int	4	Downsample ratio for segmentation masks, affecting the resolution of masks used during training.
dropout	float	0.0	Dropout rate for regularization in classification tasks, preventing overfitting by randomly omitting units during training.
val	bool	True	Enables validation during training, allowing for periodic evaluation of model performance on a separate dataset.
plots	bool	True	Generates and saves plots of training and validation metrics, as well as prediction examples, providing visual insights into model performance and learning progression.
compile	bool or str	False	Enables PyTorch 2.x <code>torch.compile</code> graph compilation with <code>backend='inductor'</code> . Accepts <code>True</code> → "default", <code>False</code> → disables, or a string mode such as "default", "reduce-overhead", "max-autotune-no-cudagraphs". Falls back to eager with a warning if unsupported.
max_det	int	300	Specifies the maximum number of objects retained during validation phase of training.



Note on Batch-size Settings

The `batch` argument can be configured in three ways:

- **Fixed Batch Size:** Set an integer value (e.g., `batch=16`), specifying the number of images per batch directly.
- **Auto Mode (60% GPU Memory):** Use `batch=-1` to automatically adjust batch size for approximately 60% CUDA memory utilization.
- **Auto Mode with Utilization Fraction:** Set a fraction value (e.g., `batch=0.70`) to adjust batch size based on the specified fraction of GPU memory usage.
- **OOM Auto-Retry:** If a CUDA out-of-memory error occurs during the first epoch, the trainer automatically halves the batch size and retries (up to 3 times). This only applies to single-GPU training; multi-GPU (DDP) training will raise the error immediately.

Augmentation Settings and Hyperparameters

Augmentation techniques are essential for improving the robustness and performance of YOLO models by introducing variability into the [training data](#), helping the model generalize better to unseen data. The following table outlines the purpose and effect of each augmentation argument:

Argument	Type	Default	Supported Tasks	Range	Description
<code>hsv_h</code>	float	0.015	detect, segment, pose, obb, classify	0.0 - 1.0	Adjusts the hue of the image by a fraction of the color wheel, introducing color variability. Helps the model generalize across different lighting conditions.
<code>hsv_s</code>	float	0.7	detect, segment,	0.0 - 1.0	Alters the saturation of the image by a

Argument	Type	Default	Supported Tasks	Range	Description
			pose, obb, classify		fraction, affecting the intensity of colors. Useful for simulating different environmental conditions.
<code>hsv_v</code>	float	0.4	detect, segment, pose, obb, classify	0.0 - 1.0	Modifies the value (brightness) of the image by a fraction, helping the model to perform well under various lighting conditions.
<code>degrees</code>	float	0.0	detect, segment, pose, obb	0.0 - 180	Rotates the image randomly within the specified degree range, improving the model's ability to recognize objects at various orientations.
<code>translate</code>	float	0.1	detect, segment, pose, obb	0.0 - 1.0	Translates the image horizontally and vertically by a fraction of the image size, aiding in learning to detect partially visible objects.
<code>scale</code>	float	0.5	detect, segment, pose, obb, classify	0 - 1	Scales the image by a gain factor, simulating objects at different distances from the camera.

Argument	Type	Default	Supported Tasks	Range	Description
<code>shear</code>	float	0.0	detect, segment, pose, obb	-180 - +180	Shears the image by a specified degree, mimicking the effect of objects being viewed from different angles.
<code>perspective</code>	float	0.0	detect, segment, pose, obb	0.0 - 0.001	Applies a random perspective transformation to the image, enhancing the model's ability to understand objects in 3D space.
<code>flipud</code>	float	0.0	detect, segment, pose, obb, classify	0.0 - 1.0	Flips the image upside down with the specified probability, increasing the data variability without affecting the object's characteristics
<code>fliplr</code>	float	0.5	detect, segment, pose, obb, classify	0.0 - 1.0	Flips the image left to right with the specified probability, useful for learning symmetrical objects and increasing dataset diversity.
<code>bgr</code>	float	0.0	detect, segment, pose, obb	0.0 - 1.0	Flips the image channels from RGB to BGR with the specified probability, useful for

Argument	Type	Default	Supported Tasks	Range	Description
					increasing robustness to incorrect channel ordering.
<code>mosaic</code>	float	1.0	detect, segment, pose, obb	0.0 - 1.0	Combines four training images into one, simulating different scene compositions and object interactions. Highly effective for complex scene understanding.
<code>mixup</code>	float	0.0	detect, segment, pose, obb	0.0 - 1.0	Blends two images and their labels, creating a composite image. Enhances the model's ability to generalize by introducing label noise and visual variability.
<code>cutmix</code>	float	0.0	detect, segment, pose, obb	0.0 - 1.0	Combines portions of two images, creating a partial blend while maintaining distinct regions. Enhances model robustness by creating occlusion scenarios.
<code>copy_paste</code>	float	0.0	segment	0.0 - 1.0	Copies and pastes objects across images to increase

Argument	Type	Default	Supported Tasks	Range	Description
					object instances.
<code>copy_paste_mode</code>	str	flip	segment	-	Specifies the copy-paste strategy to use. Options include 'flip' and 'mixup'.
<code>auto_augment</code>	str	randaugment	classify	-	Applies a predefined augmentation policy ('randaugment', 'autoaugment', or 'augmix') to enhance model performance through visual diversity.
<code>erasing</code>	float	0.4	classify	0.0 - 1.0	Randomly erases regions of the image during training to encourage the model to focus on less obvious features.
<code>augmentations</code>	list	''	detect, segment, pose, obb	-	Custom Albumentations transforms for advanced data augmentation (Python API only). Accepts a list of transform objects for specialized augmentation needs.

These settings can be adjusted to meet the specific requirements of the dataset and task at hand. Experimenting with different values can help find the optimal augmentation strategy that leads to the best model performance.

 Info

For more information about training augmentation operations, see the [reference section](#).

Logging

In training a YOLO26 model, you might find it valuable to keep track of the model's performance over time. This is where logging comes into play. Ultralytics YOLO provides support for three types of loggers - [Comet](#), [ClearML](#), and [TensorBoard](#).

To use a logger, select it from the dropdown menu in the code snippet above and run it. The chosen logger will be installed and initialized.

Comet

[Comet](#) is a platform that allows data scientists and developers to track, compare, explain and optimize experiments and models. It provides functionalities such as real-time metrics, code diffs, and hyperparameters tracking.

To use Comet:

 Example

Python

```
# pip install comet_ml
import comet_ml

comet_ml.init()
```

Remember to sign in to your Comet account on their website and get your API key. You will need to add this to your environment variables or your script to log your experiments.

ClearML

[ClearML](#) is an open-source platform that automates tracking of experiments and helps with efficient sharing of resources. It is designed to help teams manage, execute, and reproduce

their ML work more efficiently.

To use ClearML:



Example

Python

```
# pip install clearml
import clearml

clearml.browser_login()
```



After running this script, you will need to sign in to your ClearML account on the browser and authenticate your session.

TensorBoard

[TensorBoard](#) is a visualization toolkit for [TensorFlow](#). It allows you to visualize your TensorFlow graph, plot quantitative metrics about the execution of your graph, and show additional data like images that pass through it.

To use TensorBoard in [Google Colab](#):



Example

CLI

```
load_ext tensorboard
tensorboard --logdir ultralytics/runs # replace with 'runs' directory
```



To use TensorBoard locally run the below command and view results at

```
http://localhost:6006/ .
```



Example

CLI

```
tensorboard --logdir ultralytics/runs # replace with 'runs' directory
```



This will load TensorBoard and direct it to the directory where your training logs are saved.

After setting up your logger, you can then proceed with your model training. All training metrics will be automatically logged in your chosen platform, and you can access these logs to monitor your model's performance over time, compare different models, and identify areas for improvement.

FAQ

Can I train without a local GPU?

Yes. [Ultralytics Platform](#) supports cloud training with free credits to get started. Upload your dataset, select a model and GPU, and train directly from the browser. See the [cloud training guide](#) for details.

How do I train an [object detection](#) model using Ultralytics YOLO26?

To train an object detection model using Ultralytics YOLO26, you can either use the Python API or the CLI. Below is an example for both:



Single-GPU and CPU Training Example

Python

```
from ultralytics import YOLO

# Load a model
model = YOLO("yolo26n.pt") # load a pretrained model (recommended for training)

# Train the model
results = model.train(data="coco8.yaml", epochs=100, imgsz=640)
```

CLI

```
yolo detect train data=coco8.yaml model=yolo26n.pt epochs=100 imgsz=640
```

For more details, refer to the [Train Settings](#) section.

What are the key features of Ultralytics YOLO26's Train mode?

The key features of Ultralytics YOLO26's Train mode include:

- **Automatic Dataset Download:** Automatically downloads standard datasets like COCO, VOC, and ImageNet.
- **Multi-GPU Support:** Scale training across multiple GPUs for faster processing.
- **Hyperparameter Configuration:** Customize hyperparameters through YAML files or CLI arguments.
- **Visualization and Monitoring:** Real-time tracking of training metrics for better insights.

These features make training efficient and customizable to your needs. For more details, see the [Key Features of Train Mode](#) section.

How do I resume training from an interrupted session in Ultralytics YOLO26?

To resume training from an interrupted session, set the `resume` argument to `True` and specify the path to the last saved checkpoint.



Resume Training Example

Python

```
from ultralytics import YOLO

# Load the partially trained model
model = YOLO("path/to/last.pt")

# Resume training
results = model.train(resume=True)
```

CLI

```
yolo train resume model=path/to/last.pt
```

Check the section on [Resuming Interrupted Trainings](#) for more information.

How do I train a model on an imbalanced dataset?

Class imbalance occurs when some classes have significantly fewer examples than others in your training data. This can cause the model to perform poorly on rare classes. Ultralytics YOLO supports class weighting through the `cls_pw` argument to address this issue.

The `cls_pw` argument controls class weighting power based on inverse class frequency:

- `cls_pw=0.0` (default): Disables class weighting
- `cls_pw=1.0` : Applies full inverse frequency weighting
- Values between `0.0` and `1.0` : Provide partial weighting for moderate imbalance

The class weights are computed as $(1.0 / \text{class_counts}) ^ \text{cls_pw}$ and normalized so their mean equals 1.0.



Training on Imbalanced Dataset

Python

```
from ultralytics import YOLO

# Load a pretrained model
model = YOLO("yolo26n.pt")

# Train with full class weighting for severely imbalanced data
results = model.train(data="custom.yaml", epochs=100, imgsz=640, cls_pw=1.0)

# Or use partial weighting (0.25) for moderate imbalance
results = model.train(data="custom.yaml", epochs=100, imgsz=640, cls_pw=0.25)
```

CLI

```
# Train with full inverse frequency weighting
yolo detect train data=custom.yaml model=yolo26n.pt epochs=100 imgsz=640 cls_pw=1.0

# Train with partial weighting for moderate imbalance
yolo detect train data=custom.yaml model=yolo26n.pt epochs=100 imgsz=640 cls_pw=0.25
```



Tip

Start with `cls_pw=0.25` for moderately imbalanced datasets and increase to `1.0` if the rare classes still underperform. You can check the computed class weights in the training logs to verify the weight distribution.

Can I train YOLO26 models on Apple silicon chips?

Yes, Ultralytics YOLO26 supports training on Apple silicon chips utilizing the Metal Performance Shaders (MPS) framework. Specify 'mps' as your training device.